DESARROLLO WEB & BASES DE DATOS

Restaurante Álamos

Migración NoSQL a MongoDB Atlas, arquitectura modular con FastAPI y gestión integral de reservas y turnos.

CONTEXTO

Transformación Digital Álamos

Superación de la fricción operativa y sobreventas presenciales.

Problemática Inicial

El restaurante presenta dificultades en la organización y gestión de sus operaciones, principalmente en la administración de las reservas de los clientes y en la planificación de los horarios de los trabajadores. Estos procesos son gestionados de manera manual y poco centralizada, lo que dificulta mantener información actualizada y coordinada entre las distintas áreas involucradas.

Solución Implementada

Desarrollamos una plataforma web que resuelve los dos lados del restaurante: de cara al cliente, facilita la reserva de mesas y el pago en línea sin esperas; y de cara al equipo de trabajo, centraliza la gestión de turnos diarios con reglas claras para que la operación sea ordenada y eficiente.



herramientas utilizadas

Front-End Dinámico

HTML Arquitectura de la pagina web, CSS diseñador de la pagina web y JavaScript gestiona las llamadas a la API, actualiza los datos en pantalla sin recargar

Back-End FastAPI

Enrutamiento modular con APIRouter, validación estricta con Pydantic lo usamos como validador de datos en el backend: se asegura de que la información que llega desde la web como correos, contraseñas, datos de reservas o turnos venga completa y con los tipos de datos correctos antes de procesarla o guardarla en MongoDB.

MongoDB Atlas

Base de datos en la nube (MongoDB) que guarda horarios y pagos dentro de cada usuario, evitando datos repetidos.

Traspaso a Modelo relacional a MongoDB

Modelo relacional

- Tablas separadas para Usuario, Trabajador y Horarios.
- Transacciones en tablas externas con bloqueos relacionales.
- Sobrecarga de red en consultas de disponibilidad diaria.

MongoDB

- Subdocumento trabajador.horarios dentro de Usuario.
- Subdocumento transaccion embebido en Reserva.
- Lecturas completas en una sola operación atómica.

```
ESTRUCTURA DOCUMENTAL PROPUESTA (MONGODB)

usuarios
|-- identificador (RUT)
|-- datos_personales (nombre, apellido, contacto)
`-- trabajador [opcional]
    |-- credenciales (correo, clave_hash, cargo)
    `-- horarios[] [planificacion semanal, max 1/dia]

menus
|-- nombre_plato
|-- descripcion
`-- precio

reservas
|-- codigo (ej. 'RES-015', consulta publica)
|-- usuario_id (referencia a usuarios)
|-- plato (embebido: menu_id, nombre, precio historico)
|-- fecha_hora
|-- lugar (sector/mesa fisica, ej. 'Terraza 04')
|-- estado ('pendiente', 'aprobada', 'rechazada')
`-- transaccion [objeto o null]
    |-- identificador_transaccion
    |-- monto
    |-- fecha_pago
    `-- usuario_id (referencia al usuario pagador)
```

Índices de Integridad en MongoDB

```python
def inicializar_indices():
    try:
        db = get_db()
        # 1. Índice compuesto único para evitar sobreventas en la misma mesa/horario
        db.reservas.create_index(
            [("fecha", 1), ("hora", 1), ("lugar", 1)],
            unique=True,
            name="idx_reserva_fecha_hora_lugar_unique"
        )

        # Índice compuesto alternativo si se usa el campo unificado fecha_hora
        db.reservas.create_index(
            [("fecha_hora", 1), ("lugar", 1)],
            unique=True,
            sparse=True,
            name="idx_reserva_fecha_lugar_unique"
        )

        # 2. Índice único para código de reserva pública
        db.reservas.create_index(
            [("codigo", 1)],
            unique=True,
            sparse=True,
            name="idx_reserva_codigo_unique"
        )

        # 3. Índice único para el correo de trabajadores (colección usuario)
        db.usuario.create_index(
            [("trabajador.correo", 1)],
            unique=True,
            sparse=True,
            name="idx_trabajador_correo_unique"
        )
        print("✓ Índices de MongoDB Atlas inicializados con éxito.")
    except Exception as e:
        print(f"Aviso al inicializar índices de MongoDB: {e}")
```

Índice Compuesto – cero sovreventas

Bloqueo físico en MongoDB mediante `unique=True` para `(fecha, hora, lugar)`. Si dos clientes confirman al mismo milisegundo, la base de datos rechaza la colisión y la API responde con un error HTTP 409 estandarizado.

Índice Único – Acceso Seguro y Login de Personal

Evita duplicidad de credenciales y acelera el inicio de sesión del personal del restaurante.

Colección de Usuario


```
{
  "_id": {
    "$oid": "6ae968e382979c51e66e9d79"
  },
  "identificador": "33333333-3",
  "nombre": "Carlos",
  "apellido": "Muñoz",
  "contacto": "+56933333333",
  "trabajador": {
    "correo": "carlos.munoz@restaurante.cl",
    "clave_hash": "$2b$12$e8A1b2C3d4E5f6G7h8I9j0KlMnOpQrStUvWxYz",
    "cargo": "garzón",
    "horarios": [
      {
        "dia": "Lunes",
        "hora_inicio": "12:00",
        "hora_termino": "20:00"
      },
      {
        "dia": "Miércoles",
        "hora_inicio": "13:00",
        "hora_termino": "21:00"
      },
      {
        "dia": "Viernes",
        "hora_inicio": "12:00",
        "hora_termino": "20:00"
      }
    ]
  }
}
```



```
{
  "_id": {
    "$oid": "671400000000000000000002"
  },
  "identificador": "22222222-2",
  "nombre": "Diego",
  "apellido": "Muñoz",
  "contacto": "+56987654321"
}
```


Colección usuario

Esta colección almacena los datos personales de los usuarios. Cuando el usuario corresponde a un trabajador, incorpora un objeto trabajador con su información laboral y un arreglo horarios con su planificación semanal vigente.

Los clientes se almacenan en la misma colección, pero no incluyen el objeto trabajador.

Colección de reserva


```
{
  "_id": {
    "$oid": "671400000000000000000004"
  },
  "codigo": "RES-015",
  "id_reserva_anterior": 15,
  "usuario_id": {
    "$oid": "671400000000000000000002"
  },
  "plato": {
    "menu_id": {
      "$oid": "671400000000000000000003"
    },
    "nombre": "Risotto de champiñones",
    "precio_unitario": 12500
  },
  "fecha_hora": {
    "$date": "2026-06-24T20:00:00Z"
  },
  "lugar": "Terraza 04",
  "estado": "aprobada",
  "transaccion": {
    "id_transaccion_anterior": 8,
    "monto": 12500,
    "fecha_pago": "2026-06-24 20:05:00",
    "usuario_id": {
      "$oid": "671400000000000000000002"
    }
  }
}
```



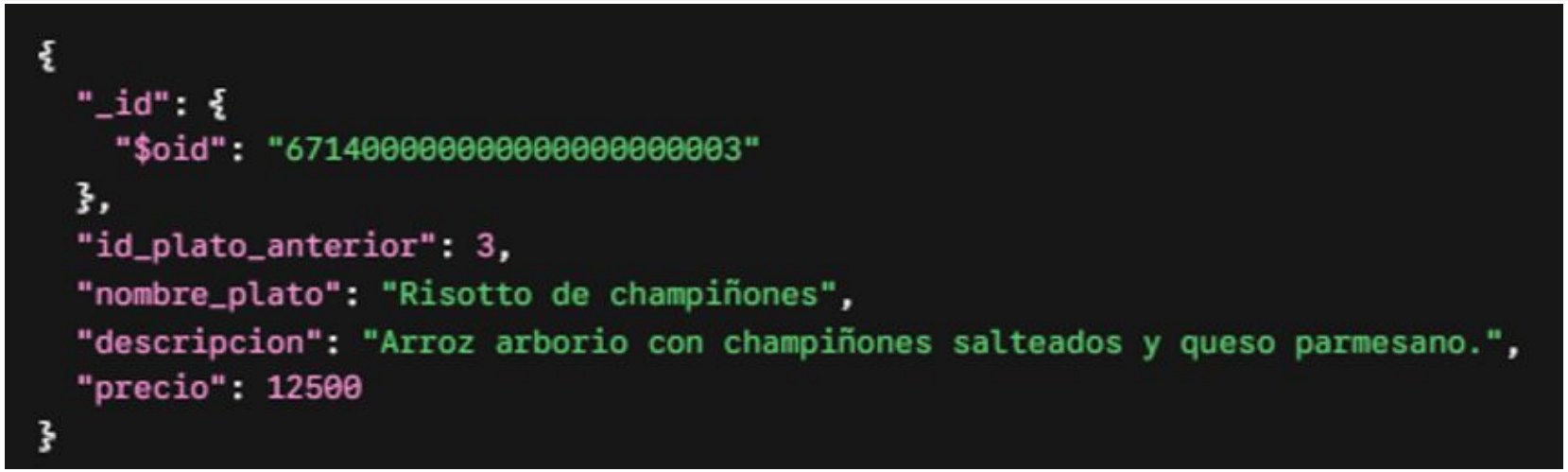
```
{
  "_id": {
    "$oid": "671400000000000000000002"
  },
  "identificador": "22222222-2",
  "nombre": "Diego",
  "apellido": "Muñoz",
  "contacto": "+56987654321"
}
```


Colección usuario

Colección reservas Esta colección almacena cada reserva como un documento independiente. Incluye el código de reserva para seguimiento público (codigo), una referencia al usuario (usuario_id)

La transacción simulada se incorpora como un objeto opcional dentro de la reserva, permitiendo registrar el pago sin generar dependencias circulares ni requerir colecciones adicionales.

Colección de Menu



```
{
  "_id": {
    "$oid": "671400000000000000000003"
  },
  "id_plato_anterior": 3,
  "nombre_plato": "Risotto de champiñones",
  "descripcion": "Arroz arborio con champiñones salteados y queso parmesano.",
  "precio": 12500
}
```

Colección usuario

Esta colección mantiene el catálogo del restaurante. Cada documento representa un plato con su nombre, descripción y precio actual.

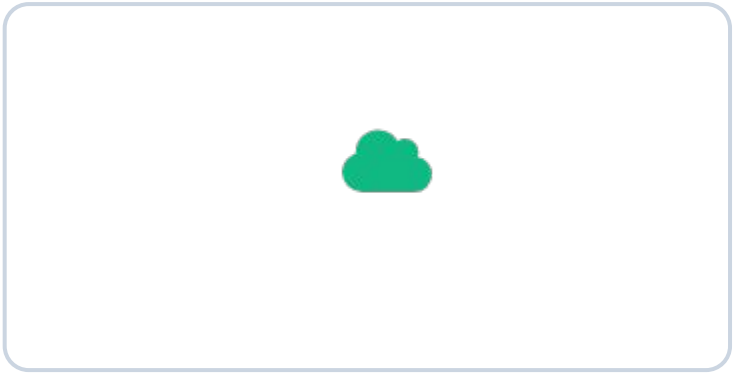
El campo _id permite identificar el plato independientemente de su nombre. El precio corresponde al valor actual del catálogo y puede modificarse sin alterar los precios históricos guardados en las reservas.

VALIDACIÓN PRÁCTICA

Paso a Demostración en Vivo







PROYECTO RESTAURANTE ÁLAMOS

¡Muchas Gracias!